

Unit 3

Dictionary Data Type

A Dictionary in Python is an **unordered**, **mutable**, and **indexed** collection of **key-value pairs**. Dictionaries are optimized for fast data retrieval and are widely used for storing structured data.

Unordered collection of items (Python 3.7+ preserves insertion order)

Mutable (can be changed after creation)

Keys must be unique and immutable (strings, numbers, tuples)

Values can be any data type (including lists, other dictionaries)

Creating Dictionaries

◇ Creating Dictionary

The simplest way to create a dictionary is by enclosing **key-value pairs** in curly braces { }

```
dictionary = {  
    "key1": "value1",  
    "key2": "value2",  
    ...  
}  
  
eg. Student={ "roll_no": 1,  
              "name": "Ibrahim",  
              "class": "6th" }
```

2. Using *dict()* Constructor

You can also create a dictionary using the *dict()* function.

Accessing Dictionary Values

Example dictionary *person* = {"name": "Alice", "age": 25, "city": "New York"}

- ◆ Printing all elements of dictionary

```
print(person)      # {'name': 'Alice', 'age': 25, 'city': 'New York'}
```

- ◆ Square Bracket Notation *[]*

```
print(person["name"]) # Output: "Alice"
```

Note it will raise an error if the key is not present in dictionary

- ◆ Safe Access with *.get()* method

```
print(person.get("age"))  # 25
```

```
print(person.get("country")) # None (instead of error)
```

```
print(person.get("country", "USA")) # Default value "USA"
```

- ◆ Accessing All Keys/Values

```
keys = person.keys()  # dict_keys(['name', 'age', 'city'])
```

```
values = person.values() # dict_values(['Alice', 25, 'New York'])
```

```
items = person.items() # dict_items([('name', 'Alice'), ...]) it stores the dictionary in a tuple
```


Modifying Dictionaries

◆ Adding/Updating Elements

Add new key-value pair

person["email"] = "alice@example.com" # Update existing key and adds a new key value pair in the dictionary

person["age"] = 26 # here it will only update the existing age

◆ Bulk Updates with .update() method

person.update({"age": 26, "phone": "123-4567"})

◆ Removing Elements

Method 1: del (raises Key Error if missing)

del person["city"]

Method 2: .pop() (returns value)

age = person.pop("age")

Method 3: .popitem() (removes last inserted item)

last_item = person.popitem() # Returns (key, value)

Method 4: Clear entire dictionary

person.clear()

Built-In Functions

Example: `dict={"a":1, "b":2}`

Function	Description	Example	Result
<code>len()</code>	Returns number of key-value pairs	<code>len(dict)</code>	2
<code>sorted()</code>	Returns sorted list of keys	<code>Sorted(dict)</code>	<code>['a', 'b']</code>
<code>any()</code>	True if any key evaluates to True	<code>any({"":1, 0:2})</code>	False
<code>all()</code>	True if all keys evaluate to True	<code>all(dict)</code>	True
<code>zip()</code>	Combine two iterables into key-value pairs: like two lists	<code>keys = ['a', 'b'] values = [1, 2] dict(zip(keys, values))</code>	<code>{'a': 1, 'b': 2}</code>
<code>min(dict)</code>	Minimum key	<code>min(dict)</code>	'a'
<code>max(dict)</code>	Maximum key	<code>max(dict)</code>	'b'
<code>sum(dict.values())</code>	Sum of values	<code>Sum(dict)</code>	3

Dictionary Methods

Example: `d={'a':1, 'b':2}`

Method	Description	Example	Result
<code>dict.keys()</code>	Returns all keys	<code>d.keys()</code>	<code>dict_keys(['a', 'b'])</code>
<code>dict.values()</code>	Returns all values	<code>d.values()</code>	<code>dict_values([1, 2])</code>
<code>dict.items()</code>	Returns key-value pairs	<code>d.items()</code>	<code>dict_items([('a', 1), ('b', 2)])</code>
<code>dict.get(key, default)</code>	Returns value of key, or default if key is missing	<code>d.get('a', 0)</code>	Value or 0
<code>dict.update(new_dict)</code>	Merges another dictionary into the current one or modifies existing value having specified key	<code>d.update({'c':3})</code>	<code>{'a':1, 'b':2, 'c':3}</code>
<code>dict.pop(key)</code>	Removes key and returns value	<code>d.pop('a')</code>	1 (removes 'a')
<code>dict.clear()</code>	Removes all items from the dictionary	<code>d.clear()</code>	<code>{}</code>
<code>copy()</code>	Returns shallow copy	<code>d2 = d.copy()</code>	New dict with same items

Tuples

Tuples are one of Python's core data structures - immutable, ordered sequences that are commonly used for storing related pieces of data together. Defined using **parentheses** `()`

Immutable: Cannot be modified after creation

Ordered: Maintain element order

Heterogeneous: Can store different data types

Indexable: Access elements by position

Hashable: Can be used as dictionary keys (if all elements are hashable)

Creating and Accessing Tuples

Creating Tuples

Empty tuple

t1 = ()

Tuple with multiple items

t2 = (1, 2, 3)

Tuple without parentheses (optional)

t3 = 1, 2, 3

Single-item tuple – comma is important!

t4 = (5,) # Valid

t5 = (5) # This is just an integer, not a tuple

Accessing Elements

print all elements of a Tuple

print(t3) # (1, 2, 3)

print(t3[0]) # 1

print(my_tuple[-1]) # 3

Basic Tuple Operations

Example: `t1=(1,2,3)`
`t2=(4,5,6)`
`t3=("a", "b")`

Operation	Example	Result
Creating Tuples	<code>Variable = ()</code> or <code>var=(1,2)</code>	Empty tuple or (1,2)
Accessing Tuple Elements	<code>print(t1)</code>	(1,2,3)
Concatenation	<code>print(t1 + t2)</code>	(1,2,3,4,5,6)
Repetition	<code>t1*3</code>	(1,2,3) (1,2,3) (1,2,3)
Membership	<code>'a' in t3</code>	True
Length	<code>Len(t1)</code>	3

Built-In Functions

Function	Description
len(t)	Returns number of items
max(t)	Returns largest item
min(t)	Returns smallest item
sum(t)	Returns sum of elements (numeric only)
sorted(t)	Returns a sorted list from the tuple
tuple()	Converts iterable into tuple

Tuple Methods

Method	Description	Example
count()	Count occurrences	t1.count(1)
index()	Find first occurrence	t1.index('2')

Tuple Packing & Unpacking

Packing means **grouping multiple values into a single tuple** — even without parentheses

Packing

person = ("Alice", 25, "Engineer") *#You don't need parentheses when packing, but you can use them for clarity:*

Unpacking means **assigning the elements of a tuple to individual variables**.

Unpacking

name, age, profession = person *#The number of variables on the left must match the number of elements in the tuple, or Python will raise an error.*

print(name) *# Alice*

print(profession) *# Engineer*

Use tuples when

- ◊ You need immutable data
- ◊ Use as **dictionary keys**
- ◊ You're working with heterogeneous data
- ◊ Multiple return values from functions
- ◊ Performance matters (tuples are slightly faster)

Sets

Sets are one of Python's built-in data types for storing **unique, unordered** collections of items. They're optimized for fast membership testing and mathematical operations. Defined using **curly braces { }**

- ◆ **Unordered:** No index positions
- ◆ **Mutable:** Can add/remove items
- ◆ **Contains only unique elements:** Duplicates are automatically removed
- ◆ **Can contain only hashable types:** Numbers, strings, tuples (not lists/dicts)
- ◆ Sets can **only contain immutable types** (e.g., no lists/dictionaries inside)

Main uses of sets

- ◆ Removing duplicates from a list
- ◆ Finding common friends
- ◆ Finding elements in one set but not another

Basic Operations

Adding Elements

.add()

.update()

Removing Elements

.remove()

.discard()

.pop()

.clear()

Membership Testing

in

Union (all elements from both sets)

set1.union(set2)

Intersection (common elements)

set1.intersection(set2)

Difference (in a but not b)

set1.difference(set2)

Comparison Operations

Subset Check

s1.issubset(s2)

OR

s1 <= s2

Superset Check

s1.issuperset(s2)

OR

s1 >= s2

Disjoint Check (No Common Elements)

s1.isdisjoint(s2)

Adding Elements

.add() Add Single Element

```
fruits = {"apple", "banana"}  
fruits.add("orange")  
print(fruits) # {'apple', 'banana', 'orange'}
```

.update() Add Multiple Elements

```
fruits.update(["kiwi", "mango"])  
print(fruits) # {'apple', 'banana', 'orange', 'kiwi', 'mango'}
```

Removing Elements

.remove() Remove Specific Element (Raises Error if Missing)

```
fruits.remove("banana")  
print(fruits) # {'apple', 'orange', 'kiwi', 'mango'}
```

.discard() Remove Element (No Error if Missing)

```
fruits.discard("pear") # No error even though pear doesn't exist
```

.pop() Remove and Return Arbitrary Element

```
removed = fruits.pop()  
print(f'Removed {removed}') # Removes and returns one element
```

.clear() Remove All Elements

```
fruits.clear()  
print(fruits) # set()
```

Membership Testing

In Operator - Check if Element Exists

```
colors = {"red", "green", "blue"}  
print("red" in colors) # True  
print("yellow" not in colors) # True
```

Set Operations

Union (all elements from both sets)

```
a = {1, 2, 3}  
b = {3, 4, 5}  
# Method union_  
set = a.union(b) # {1, 2, 3, 4, 5}  
# Operator  
union_set = a | b # {1, 2, 3, 4, 5}
```

Intersection (common elements)

```
# Method  
intersect_set = a.intersection(b) # {3}  
# Operator  
intersect_set = a & b # {3}
```

Difference (in a but not b)

```
# Method  
diff_set = a.difference(b) # {1, 2}  
# Operator  
diff_set = a - b # {1, 2}
```

Comparison Operations

Subset Check

```
x = {1, 2} y = {1, 2, 3}
```

```
print(x.issubset(y)) # True
```

```
print(x <= y)      # True (operator version)
```

Superset Check

```
print(y.issuperset(x)) # True
```

```
print(y >= x)       # True (operator version)
```

Disjoint Check (No Common Elements)

```
z = {4, 5}
```

```
print(x.isdisjoint(z)) # True
```

Key Takeaways

- ◇ Sets automatically handle uniqueness - no duplicates allowed
- ◇ Use *add()* for single elements, *update()* for multiple
- ◇ *remove()* raises errors for missing elements, *discard()* doesn't
- ◇ Mathematical operations have both method (*union()*) and operator (*|*) forms
- ◇ Sets are optimized for membership testing (*O(1)* time complexity)
- ◇ Use sets when you need to ensure uniqueness or perform set operations

Traversing of Sets

Basic Methods for Traversing Sets

◆ 1. Simple For Loop (Most Common)

```
colors = {"red", "green", "blue"}  
for color in colors:  
    print(color)
```

◆ 2. While Loop with pop()

```
while colors:  
    current = colors.pop()  
    print(f"Processing {current}")
```

◆ 3. Using Enumerate (Get Counter)

```
for index, color in enumerate(colors):  
    print(f"Color {index + 1}: {color}")
```

Important Notes About Set Traversal

- ◆ **No Index Access:** Cannot use `colors[0]` Type-Error
- ◆ **Unordered:** Elements may appear in any order (though Python 3.7+ preserves insertion order)
- ◆ **Modification During Iteration:** Generally unsafe, may raise Runtime-Error
- ◆ **Performance:** $O(n)$ time complexity for full traversal

Built In Functions

```
s = {5, 2, 9, 1}
```

```
print(len(s))    # 4      Returns the number of elements in the set
```

```
print(max(s))    # 9      Returns the largest element
```

```
print(min(s))    # 1      Returns the smallest element
```

```
print(sum(s))    # 17     Returns the sum of all elements (numeric sets only)
```

```
print(sorted(s)) # [1, 2, 5, 9]  #sorted(s) always returns a list, not a set.
```

```
print(all(s))    # True (since all are non-zero)
```

```
print(any(s))    # True (at least one non-zero)
```

```
print(type(s))   # <class 'set'>
```

```
e=enumerate(s)  # [(0, 5), (1, 2),(2,9),(3,1)]  Returns an iterator that yields pairs of index and value
```

Note `max()`, `min()`, and `sum()` work only if elements are **numeric**.

Set Methods

Example: `s={1,2}`

Method	Description	Example	Result
<i>add(element)</i>	Adds a single element	<i>s.add(3)</i>	{1, 2, 3}
<i>update(*iterables)</i>	Adds multiple elements	<i>s.update([3,4])</i>	{1, 2, 3, 4}
<i>remove(element)</i>	Removes element (raises KeyError if missing)	<i>s.remove(2)</i>	{1, 3}
<i>discard(element)</i>	Removes element (no error if missing)	<i>s.discard(5)</i>	{1, 2, 3}
<i>pop()</i>	Removes and returns arbitrary element	<i>s.pop()</i>	Returns 1
<i>clear()</i>	Removes all elements	<i>s.clear()</i>	set()
<i>copy()</i>	Returns shallow copy	<i>s.copy()</i>	

Strings

A string is a sequence of characters enclosed in quotes. **Strings are immutable** (cannot be modified in-place).

Python supports:

Single quotes: 'Hello'

Double quotes: "Hello"

Triple quotes (for multiline strings): `"""Hello"""` or `"""Hello"""`.

Creating and Storing Strings

```
str1 = 'Hello'
```

```
str2 = "Python"
```

```
str3 = '''This is a  
multiline string'''
```

Basic String Operations

- ◆ Concatenation (Joining Strings)
- ◆ Repetition (Multiplying Strings)
- ◆ Finding String Length (len())
- ◆ Accessing Characters by Index
- ◆ String Slicing (Extracting Substrings)
- ◆ Checking Substrings (in and not in)
- ◆ String Methods for Case Conversion
- ◆ Trimming Whitespace
- ◆ Replacing Substrings
- ◆ Splitting & Joining Strings

Operation	Example	Result
Concatenation	<code>"A" + "B"</code>	<code>"AB"</code>
Repetition	<code>"Hi" * 3</code>	<code>"HiHiHi"</code>
Length	<code>len("ABC")</code>	<code>3</code>
Indexing	<code>"ABC"[1]</code>	<code>'B'</code>
Slicing	<code>"Python"[1:4]</code>	<code>'yth'</code>
Substring Check	<code>"Hi" in "Hi!"</code>	<code>True</code>
Case Change	<code>"Hi".upper() "HI".lower()</code>	<code>"HI" "hi"</code>
Replace	<code>"A B".replace(" ", "-")</code>	<code>"A-B"</code>
Split	<code>"A,B".split(",")</code>	<code>["A", "B"]</code>
Join	<code>"-".join(["A", "B"])</code>	<code>"A-B"</code>

String Slicing

1. String Slicing (Extracting Substrings)

Slicing allows you to extract a portion of a string using the syntax:

string[start : stop : step]

- ◇ *start*: Index where slicing begins (inclusive, default=0)
- ◇ *stop*: Index where slicing ends (exclusive, default=len(string))
- ◇ *step*: Interval between characters (default=1)

Text = "Python Programming"

Operation	Example	Result	Explanation
Basic slice	<code>text[0:6]</code>	"Python"	Chars from index 0 to 5
Omit start	<code>text[:6]</code>	"Python"	From start to index 5
Omit stop	<code>text[7:]</code>	"Programming"	From index 7 to end
Negative indices	<code>text[-11:-1]</code>	"Programmin"	Chars from -11 to -2
Step of 2	<code>text[::-2]</code>	"Pto rgamn"	Every 2nd character
Reverse string	<code>text[::-1]</code>	"gnimmargorP nohtyP"	Full string backwards

String Joining

2. String Joining (Combining Strings)

Python provides two main ways to join strings:

+ Operator (Concatenation)

```
str1 = "Hello"
```

```
str2 = "World"
```

```
result = str1 + " " + str2    # "Hello World"
```

Limitation: Inefficient for joining many strings

join() Method (Recommended)

Much more efficient for joining multiple strings

The string before *join()* is used as the separator | `sentence = " ".join(words)`

Comma-separated values

```
tags = ["python", "coding", "tutorial"]  
csv_line = ",".join(tags) # "python,coding,tutorial"
```

Path joining

```
parts = ["home", "user", "documents"]  
path = "/".join(parts) # "home/user/documents"
```

Multicharacter separator

```
joined = "->".join(["A", "B", "C"]) # "A -> B -> C"
```

String Methods

Method	Description	Example	Result
str.lower()	Convert to lowercase	"HELLO".lower()	"hello"
str.upper()	Convert to uppercase	"hello".upper()	"HELLO"
str.strip()	Remove whitespace from both ends	" hi ".strip()	"hi"
str.split()	Split into list by separator (default: space)	"a,b,c".split(",")	['a','b','c']
str.join()	Join list into string with separator	" ".join(["Hello","World"])	"Hello World"
str.replace(old, new)	Replace occurrences of substring	"hello".replace("l","x")	"hexxo"
str.startswith(prefix)	Check if starts with prefix	"file.txt".startswith("file")	True
str.endswith(suffix)	Check if ends with suffix	"data.csv".endswith(".csv")	True
str.find(sub)	Find index of substring (-1 if not found)	"python".find("th")	2
str.count(sub)	Count occurrences of substring	"banana".count("a")	3

Formatting Strings

◇ 1. f-Strings (Python 3.6+) Most Modern & Recommended

```
name = "Alice"
age = 25
# Basic f-string
print(f"My name is {name} and I'm {age} years old")
# Expressions inside f-strings
print(f"Next year I'll be {age + 1}")
# Formatting numbers
price = 19.99
print(f"Price: {price:.1f}") # "Price: 19.9"
# Alignment
text = "Hello"
print(f'{text:>10}') # Right-aligned: "    Hello"
print(f'{text:^10}') # Center-aligned: "  Hello  "
```

◇ 2. str.format() Method - Flexible Alternative

```
# Positional arguments
print("Hello {}, you're {} years old".format(name, age))
# Named arguments
print("Hello {name}, you're {age} years old".format(name="Bob", age=30))
```

3. %-formatting - Legacy Method

```
# Basic substitution
print("Hello %s, you're %d years old" % (name, age))
# Number formatting
print("Percentage: %.1f%%" % 99.5) # "Percentage: 99.5%"
```

4. Template Strings (Safe for User Input)

```
from string import Template
t = Template("Hello $name, you're $age years old")
print(t.substitute(name="Charlie", age=40))
```


Use for?

- ◇ **f-strings** - For most cases (cleanest syntax, fastest)
- ◇ **str.format()** - When you need reusable templates
- ◇ **%-formatting** - Only for maintaining legacy code
- ◇ **Template strings** - When security is a concern

Files

Files in Python are objects that allow your programs to interact with external data stored on your computer's storage system.

They provide a way to persistently store information beyond a program's runtime and access data from the operating system.

Core Concepts

1. What is a File?

- ◇ A file is a named location on disk that stores related information. In Python, files are treated as **file objects** that can be manipulated using file handling functions.

2. File Handling Basics

Python performs file operations in three main steps:

- ◇ **Opening** the file
- ◇ **Reading** from or **writing** to the file
- ◇ **Closing** the file

Types of Files

1. Text Files

Human-readable files containing plain text.

Store data as characters

Use encoding (typically UTF-8)

Can be edited with basic text editors

Newline characters are automatically converted

Common Extensions:

.txt - Plain text files

.csv - Comma-separated values

.json - JavaScript Object Notation

.xml - Extensible Markup Language

.html/.htm - HyperText Markup Language

2. Binary Files

Non-text files storing raw byte data.

No character encoding

Preserve exact byte representation

Require special programs to interpret

Newlines are not converted

Common Extensions:

.jpg, .png - Image files

.mp3, .wav - Audio files

.mp4, .avi - Video files

.exe, .dll - Executable files

.pdf - Portable Document Format

Access Modes .

1. Basic File Modes

Mode	Description	Pointer Position	If File Exists	If File Doesn't Exist
'r'	Read (default)	Start	Opens	Error (FileNotFoundError)
'w'	Write	Start	Overwrites	Creates new
'a'	Append	End	Preserves content	Creates new
'x'	Exclusive creation	-	Error (FileExistsError)	Creates new

2. Combined Modes

Mode	Description	Pointer Position	If File Exists	If File Doesn't Exist
'r+'	Read + Write	Start	Opens	Error
'w+'	Write + Read	Start	Overwrites existing content	Creates new file
'a+'	Append + Read	End (write), Start (read)	Preserves content, appends new data	Creates new file

3. Binary Modes

'b' flag (e.g., 'rb', 'wb', 'ab', 'rb+', 'wb+', 'ab+') rest of the features are same as above, we just have to add b with other modes

Operations & Methods used on Files

1. File Opening Methods

open()

Basic syntax

file = open('filename.txt', 'mode')

Recommended approach

with open('filename.txt', 'r') as file:

File operations

file.close()

close()

2. Reading Methods

read()

Reads entire file content

content = file.read()

readline()

Reads single line

line = file.readline()

readlines()

Returns list of all lines

lines = file.readlines()

3. Writing Methods

write()

Writes string to file

file.write("text")

writelines()

Writes list of strings

file.writelines(list_of_lines)

4. Position Control Methods

seek()

Moves file pointer

file.seek(10)

tell()

Returns current position

pos = file.tell()

Examples on Access Modes ..

1. Basic File Modes

Read mode

```
with open('file.txt', 'r') as f:  
    content = f.read()
```

Write mode (overwrites)

```
with open('file.txt', 'w') as f:  
    f.write("New content")
```

Append mode

```
with open('file.txt', 'a') as f:  
    f.write("\nAdded line")
```

Exclusive creation

```
try:  
    with open('newfile.txt', 'x') as f:  
        f.write("Brand new file")  
except FileExistsError:  
    print("File already exists")
```

2. Combined Modes

Read and write

```
with open('data.txt', 'r+') as f:  
    content = f.read()  
    f.write("\nAdditional data")
```

Write and read (truncates first)

```
with open('data.txt', 'w+') as f:  
    f.write("New content")  
    f.seek(0) # Move to start  
    print(f.read())
```

Append and read

```
with open('logs.txt', 'a+') as f:  
    f.write("\nNew log entry")  
    f.seek(0)  
    logs = f.readlines()
```

Creating and Reading Text Data

1. Creating Text Files

Basic File Creation

```
# Create new file (overwrites if exists)  
with open('example.txt', 'w', encoding='utf-8') as file:  
    file.write("First line of text\n")  
    file.write("Second line with number: %d\n" % 42)  
    file.write(f"Formatted line: {3.14:.2f}\n")
```

Writing Multiple Lines Efficiently

```
lines = [ "Line 1\n", "Line 2\n", "Line 3\n"]  
# More efficient than multiple write() calls  
with open('multiline.txt', 'w') as file:  
    file.writelines(lines)
```

Appending to Existing Files

```
with open('log.txt', 'a') as file:    # 'a' for append mode  
    file.write("New log entry at: " + str(datetime.now()) + "\n")
```

2. Reading Text Files

Reading Entire Content

```
with open('document.txt', 'r') as file:  
    content = file.read() # Returns single string  
    print(content)
```

Reading Line by Line (Memory Efficient)

```
with open('large_file.txt', 'r') as file:  
    for line in file: # Reads one line at a time  
        # Remove whitespace/newlines  
        processed_line = line.strip()  
        print(processed_line)
```

Reading Specific Number of Characters

```
with open('data.txt', 'r') as file:  
    # Reads first 100 characters  
    first_100_chars = file.read(100)  
    Print(first_100_chars)
```

Why Files Matter in Python

Data Persistence:

Save program state between runs

Configuration:

Store user settings and preferences

Data Processing:

Handle large datasets

Interoperability:

Exchange data with other programs

Logging:

Record program execution details